



Python for Beginners with applications

Lesson 7:

*Write code from scratch in a clear &
concise way, with a complete basic
course.*

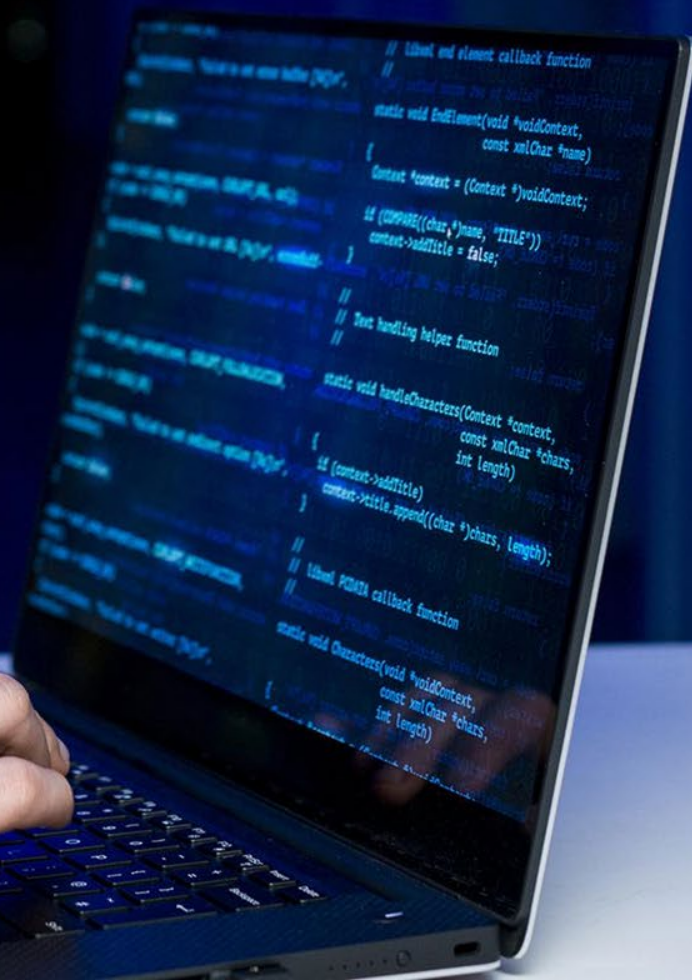
From beginners to intermediate.

*Further data visualization
NumPy library
Introduction to Pandas library*

Abdesselam Filali 2025-05-27 18h30 Algiers time

1.

Further Data Visualisation



Matplotlib: The foundational plotting library, used for basic charts.

Seaborn: Built on top of Matplotlib, ideal for statistical plots.

Plotly: Used for interactive and web-based visualizations.

The Matplotlib logo features the word "matplotlib" in a blue sans-serif font. The "o" is replaced by a circular icon containing a stylized multi-colored pie chart with segments in orange, yellow, green, and blue.

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

The Seaborn logo consists of a circular icon on the left depicting a blue landscape with white mountains and a bar chart. To the right of the icon, the word "seaborn" is written in a blue sans-serif font.

<https://seaborn.pydata.org/>

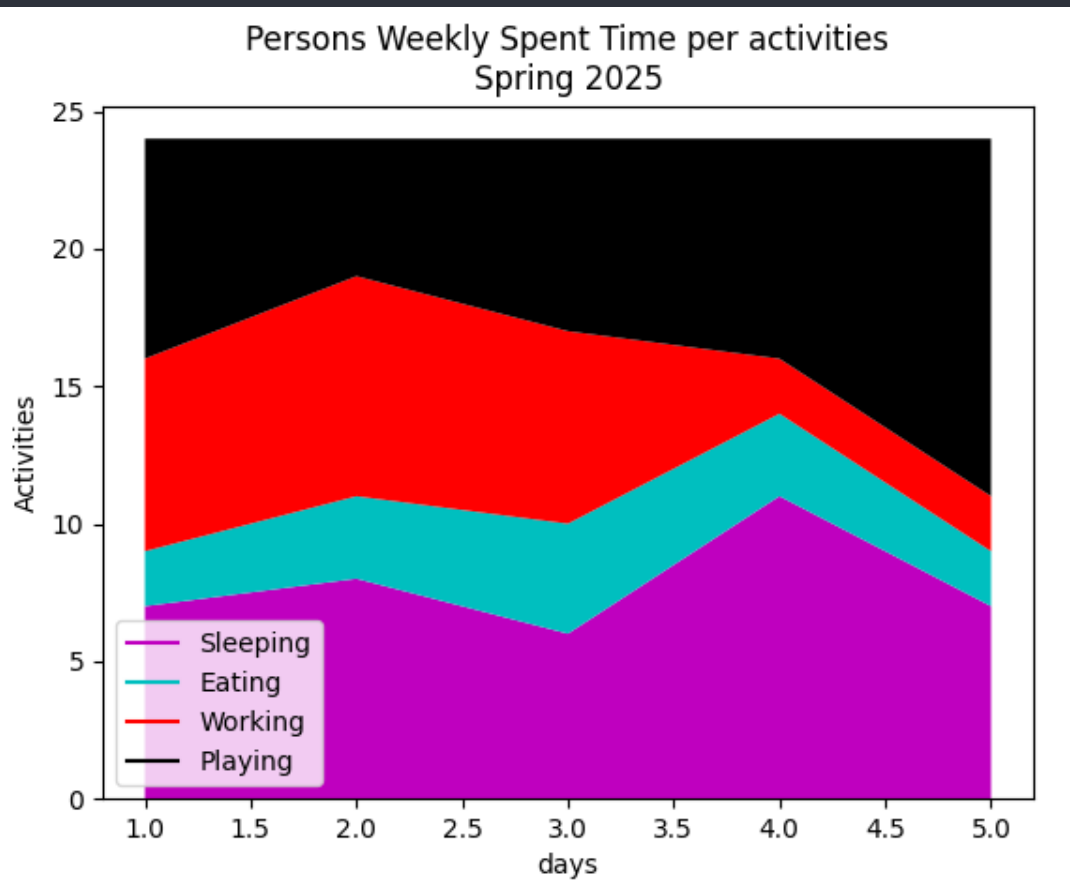
The Plotly logo features a stylized icon on the left made of three vertical bars of varying heights, each topped with a colored dot (pink, purple, and blue). To the right of the icon, the word "plotly" is written in a black sans-serif font.

<https://plotly.com/python/>

Use Case: Stack plots present the frequency of every activity, such as the frequency of sleeping, eating, working, and playing per day. In this data set, on day 2, a person spent eight hours sleeping, three hours in eating, eight hours working, and five hours playing.

```
course6 > 4_08_Stack plot.py > ...
```

```
1  import matplotlib.pyplot as plt
2  days = [1, 2, 3, 4, 5]
3  sleeping = [7, 8, 6, 11, 7]
4  eating = [2, 3, 4, 3, 2]
5  working = [7, 8, 7, 2, 2]
6  playing = [8, 5, 7, 8, 13]
7  plt.plot([],[], color='m', label='Sleeping')
8  plt.plot([],[], color='c', label='Eating')
9  plt.plot([],[], color='r', label='Working')
10 plt.plot([],[], color='k', label='Playing')
11 plt.stackplot(days, sleeping, eating, working ,
12 playing, colors=['m','c', 'r', 'k'])
13 plt.xlabel('days')
14 plt.ylabel('Activities')
15 plt.title('Persons Weekly Spent Time per activities\nSpring 2025')
16 plt.legend()
17 plt.show()
```

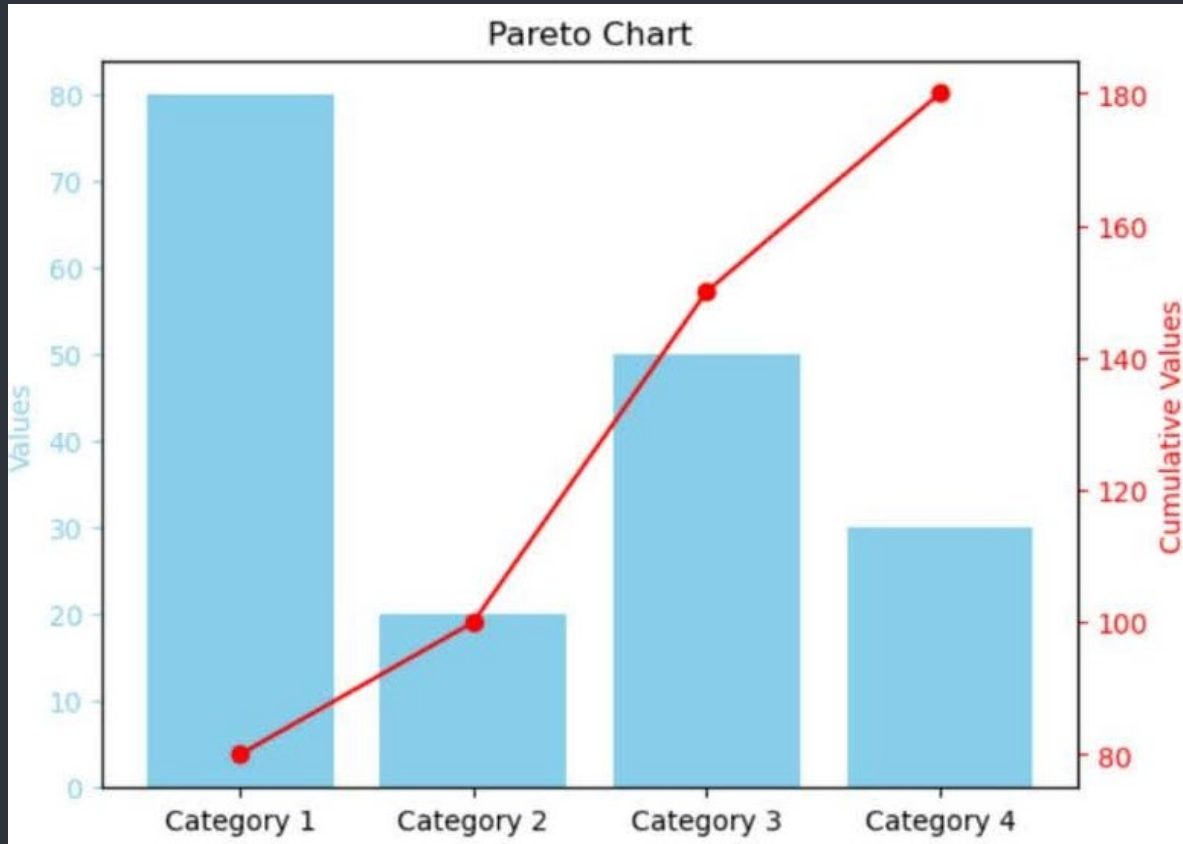


Use Case: Categories and probability distribution.

```
1 import matplotlib.pyplot as plt
2 categories = ['Category 1', 'Category 2', 'Category 3', 'Category 4']
3 values = [80, 20, 50, 30]
4 fig, ax1 = plt.subplots()
5 ax1.bar(categories, values, color='skyblue')
6 ax1.set_ylabel('Values', color='skyblue')
7 ax1.tick_params('y', colors='skyblue')
8 ax2 = ax1.twinx()
9 cumulative_values = [sum(values[:i+1]) for i in range(len(values))]
10 ax2.plot(categories, cumulative_values, color='red', marker='o')
11 ax2.set_ylabel('Cumulative Values', color='red')
12 ax2.tick_params('y', colors='red')
13 plt.title('Pareto Chart')
14 plt.show()
```

Stack plot

Library : matplotlib

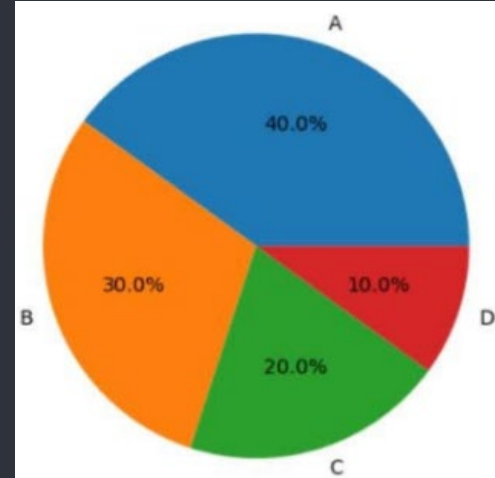




Use Case: Showing the distribution of data.

```
course6 > 4_06_Pie.py > ...
```

```
1  import matplotlib.pyplot as plt
2  sizes = [40, 30, 20, 10]
3  labels = ['A', 'B', 'C', 'D']
4  plt.pie(sizes, labels=labels, autopct='%1.1f%%')
5  plt.show()
```



Donut chart

Library : matplotlib



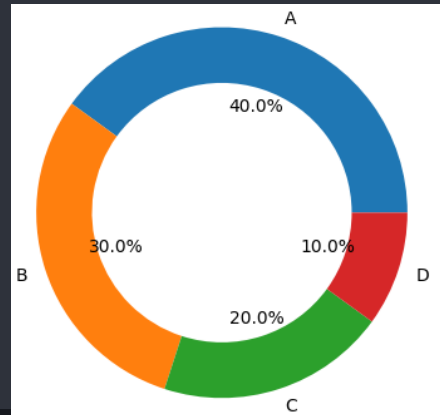
Use Case: Showing the distribution of data.

```
course6 > 4_07_Donut.py > ...
```

```
1 import matplotlib.pyplot as plt
2 sizes = [40, 30, 20, 10]
3 labels = ['A', 'B', 'C', 'D']
4 plt.pie(sizes, labels=labels, autopct='%1.1f%%', wedgeprops={'width':0.3})
5 plt.show()
```

<https://python-graph-gallery.com/donut-plot/>

https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_and_donut_labels.html





Use Case: How to customize Pie charts using matplotlib.

```
course6 > 4_06_PieCustomize1.py > ...
```

```
1  age_group_labels = [  
2      'Children 0-18', 'Adults 19-25', 'Adults 26-34',  
3      'Adults 35-54', 'Adults 55-64', 'Seniors 65+'  
4  ]  
5  age_group_populations = [  
6      75307800, 27799100, 39817700,  
7      81478600, 42061700, 52784400  
8  ]  
9  # print age groups and corresponding populations  
10 for label, population in zip(age_group_labels, age_group_populations):  
11     print (f'{label}:    {population:,}')
```

2.

NumPy Library



What is NumPy?

NumPy (Numerical Python) is a fundamental powerful library for mathematical and numerical operations.

It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these arrays efficiently.

Why use NumPy?

- Much faster than Python lists for numerical tasks
- Used as the foundation for libraries like Pandas, TensorFlow, and Scikit-learn
- Makes matrix operations, statistics, and data transformations easy and efficient

installation

```
pip install numpy
```



<code>A[0,0], A[0,1], A[0,2]</code>	1	2	3
<code>A[1,0], A[1,1], A[1,2]</code>	5	6	7
<code>A[2,0], A[2,1], A[2,2]</code>	8	9	10
<code>A[3,0], A[3,1], A[3,2]</code>	11	12	13

What is NumPy Array?

A NumPy array is a powerful data structure that stores elements of the same data type in a grid-like format. It's much faster and more efficient than Python lists for numerical operations..

Creating Arrays:

```
import numpy as np

np.array([1, 2, 3])           # From a list
np.zeros((2, 3))              # Array of zeros
np.ones((3, 3))               # Array of ones
np.arange(0, 10, 2)           # Range of numbers
```

Creating Array:

Code:

```
4_01_numpy.py > ...
1  import numpy as np
2  arr = np.array([1, 2, 3, 4, 5]) # Creating a NumPy array
3  result = arr * 2 # Performing mathematical
4  |         |         |         |         | # operations on the array
5  print(result)      #output [2 4 6 8 10]
```

Basic Array Operations:

Indexing and slicing:

```
arr = np.array([10, 20, 30, 40])  
arr[1]          # 20  
arr[1:3]        # [20 30]
```

Array Shape & Dimensions:

```
arr.shape        # Returns (rows, columns)  
arr.ndim         # Returns number of dimensions  
arr.size         # Total number of elements
```

Reshaping Arrays:

```
arr.dtype        # Get the data type of elements
```


Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])
```

```
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])
```

```
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

What is NumPy?

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.

NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

NumPy stands for Numerical Python.

Why Use NumPy?

In Python we have lists that serve the purpose of arrays, but they are slow to process.

NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.

The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.

Arrays are very frequently used in data science, where speed and resources are very important.

Why is NumPy Faster Than Lists?

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

This behavior is called locality of reference in computer science.

This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

Why is NumPy Faster Than Lists?

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

This behavior is called locality of reference in computer science.

This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

```
1  NumPy installation
2      import numpy
3  example :
4      import numpy
5      arr = numpy.array([1, 2, 3, 4, 5])
6      print(arr)
7  NumPy as np
8      import numpy as np
9  example :
10     import numpy as np
11     arr = np.array([1, 2, 3, 4, 5])
12     print(arr)
13  Checking version
14     print(np.__version__)
```

Creating a NumPy ndarray object :

NumPy is used to work with arrays. The array object in NumPy is called ndarray.

We can create a NumPy ndarray object by using the `array()` function.

Example :

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))
```


Let's use NumPy in a real-world scenario: analyzing temperature data!
Example: Calculate Average Weekly Temperature

4_05_CalculateAvgWeeklyTemp.py > ...

```
1  import numpy as np
2
3  # temperature in degrees celsius for 7 days
4  temps=np.array([29.3, 42.1, 18.8, 22.1, 17.6, 28.7, 33.4])
5
6  #average temperature
7  avg_temp=np.mean(temps)
8  print(f"Average temperature for 7 days: {avg_temp:.1f} degrees celsius")
9
10 #highest temperature
11 max_temp=np.max(temps)
12 print(f"Highest temperature for 7 days: {max_temp:.1f} degrees celsius")
13
14 #lowest temperature
15 min_temp=np.min(temps)
16 print(f"Lowest temperature for 7 days: {min_temp:.1f} degrees celsius")
```

#1 **np.array()**: Create a NumPy array from a Python list or tuple.

#2 **np.zeros()**: Create an array filled with zeros of a specified shape.

#3 **np.ones()**: Create an array filled with ones of a specified shape.

#4 **np.arange()**: Create an array with values within a specified range.

#5 **np.linspace()**: Create an array with evenly spaced values over a specified interval.

#6 **np.reshape()**: Reshape an array into a new shape.

#7 **np.random.rand()**: The rand() function is used to generate an array with random values between 0 to 1.

#8 **np.random.randn()**: Generate an array of random numbers from a standard normal distribution. (close to zero)

#9 **np.random.randint()**: Generate an array of random integers within a specified range.

#10 **np.mean()**: Calculate the mean of array elements.

#11 **np.median()**: Calculate the median of array elements.

- #12 **np.std()**: Calculate the standard deviation of array elements.
- #13 **np.sum()**: Compute the sum of array elements.
- #14 **np.min(), np.max()**: Find the minimum and maximum values in an array.
- #15 **np.argmin(), np.argmax()**: Find the indices of the minimum and maximum values in an array.
- #16 **np.dot()**: Compute the dot product of two arrays.
- #17 **np.transpose()**: Calculate transpose of the array.
- #18 **np.concatenate()**: Concatenate arrays along a specified axis.

#19 **np.split()**: Split an array into multiple sub-arrays.

#20 **np.vstack()**, **np.hstack()**: Stack arrays vertically and horizontally.

#21 **np.unique()**: Find the unique elements in an array.

#22 **np.save()**, **np.load()**: Save and load arrays to/from disk.

#23 **np.clip()**: Clip (limit) the values in an array.

#24 **np.where()**: Return elements chosen from two arrays based on a condition.

#25 **np.linalg.inv()**: Calculate inverse of the matrix.

- #26 **np.linalg.det()**: Calculate determinant of the matrix.
- #27 **np.linalg.solve()**: Solve a system of linear equations.
- #28 **np.percentile()**: Compute the nth percentile of the data.
- #29 **np.corrcoef()**: Compute the correlation coefficient between two arrays.
- #30 **np.deg2rad()**, **np.rad2deg()**: Convert angles from degrees to radians and vice versa.
- #31 **np.argsort()**: Return the indices that would sort an array.
- #32 **np.searchsorted()**: Find indices where elements should be inserted to maintain order.

#33 `np.arcsin()`, `np.arccos()`, `np.arctan()`:
Inverse trigonometric functions.

#34 `np.linalg.eig()`: Eigenvalues and
eigenvectors of a square matrix.

#35 `np.linalg.svd()`: Singular value
decomposition.

3.

Introduction to Pandas library



1 **What is Pandas?**

2 pandas is a fast, powerful, flexible and easy to use open source data
3 analysis and manipulation tool,
4 built on top of the Python programming language.

5 **Why use pandas?**

6 Pandas allows us to analyze big data and make conclusions based on
7 statistical theories.
8 Pandas can clean messy data sets, and make them readable and relevant.
9 Relevant data is very important in data science.

10 **installation**

11 pip install pandas

12
13
14
<https://www.w3schools.com/python/pandas/default.asp>
<https://pandas.pydata.org/>

1 What Can Pandas Do?

2 Pandas gives you answers about the data. Like:

3 Is there a correlation between two or more columns?

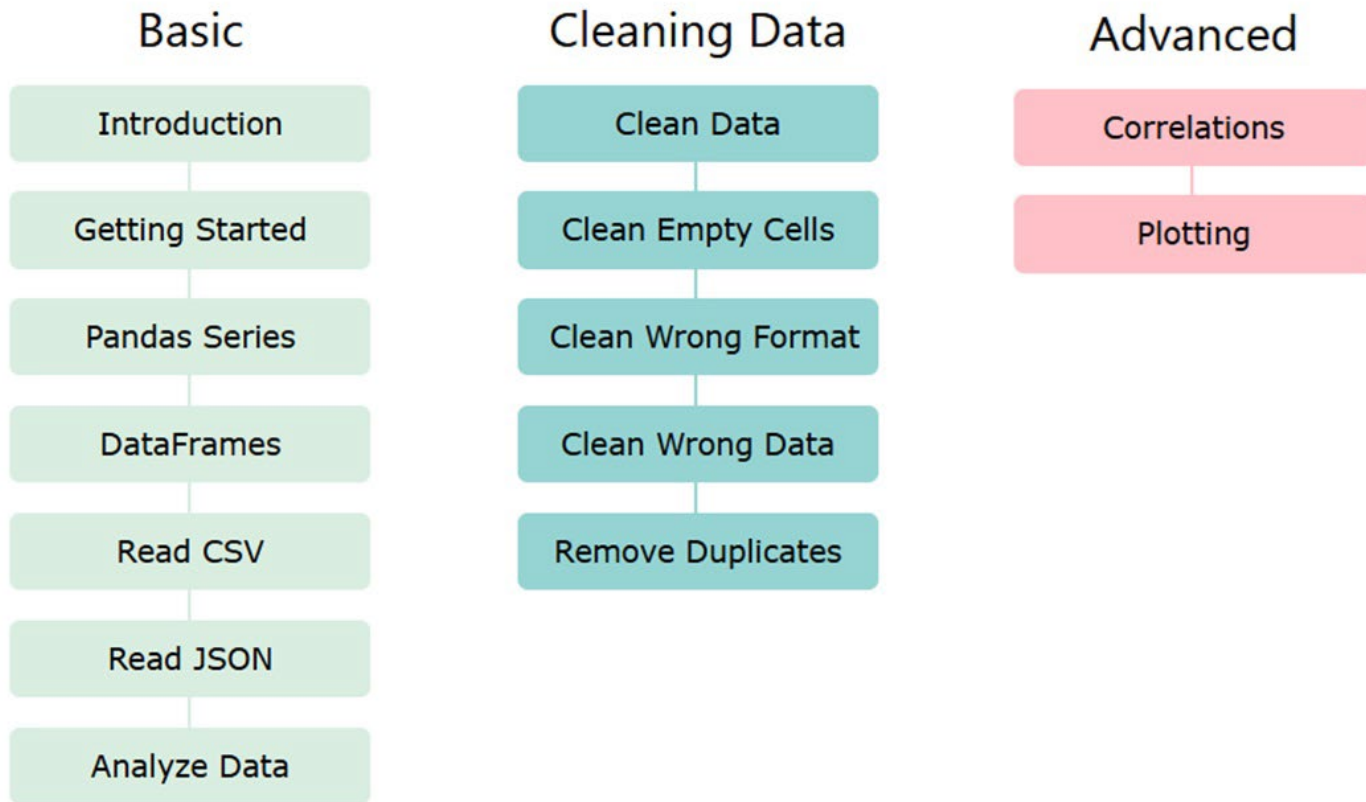
4 What is average value?

5 Max value?

6 Min value?

7 Pandas are also able to delete rows that are not relevant, or contains
8 wrong values, like empty or NULL values. This is called cleaning the
9 data.

1
2
3
4
5
6
7
8
9
10
11
12
13
14



```
1 Import pandas
2     import pandas
3
4 example:
5
6     import pandas
7     mydataset = {
8         'cars': ["BMW", "Volvo", "Ford"],
9         'passings': [3, 7, 2]
10    }
11    myvar = pandas.DataFrame(mydataset)
12    print(myvar)
13
14 pandas as pd import pandas as pd
15
16 Checking pandas version    print(pd.__version__)
```